

Lab 2: Multilayer Perceptron (MLP) and Hyperparameter Tuning

Lab Objectives

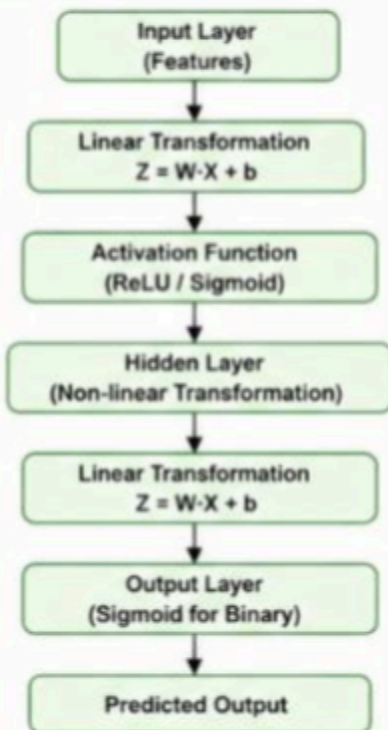
- Implement a Multilayer Perceptron (MLP) with at least one hidden layer using Keras or PyTorch. Analyze the impact of activation functions (Sigmoid vs. ReLU) on convergence.
-
- Observe the relationship between learning rates and model performance.

1. Logic Flowchart: Forward and Backward Pass

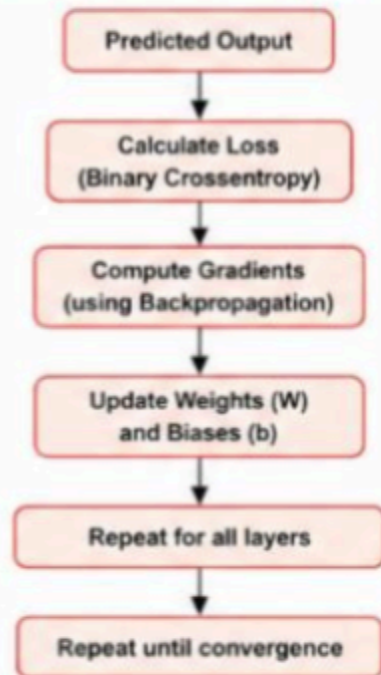
Before coding, describe the mathematical and logical flow of data through your MLP. In the space below, outline the steps for the **Forward Pass** (input to output) and the **Backward Pass** (gradient calculation and weight update via backpropagation).

Logic flowchart description

Forward Pass



Backward Pass (Backpropagation)



2. Implementation Task

Construct an MLP to solve a non-linearly separable problem (e.g., the XOR gate or the Iris dataset). Your architecture must include:

- An input layer matching the feature dimension.
- At least one hidden layer with a configurable number of neurons.
- An output layer with an appropriate activation (e.g., Softmax for multi-class, Sigmoid for binary).

```

import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train = X_train / 255.0
X_test = X_test / 255.0

print("Training Images Shape:", X_train.shape)
print("Testing Images Shape:", X_test.shape)

mlp_model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

mlp_model.summary()

mlp_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("\n--- Training MLP ---")

history = mlp_model.fit(
    X_train,
    y_train,
    epochs=10,
    validation_split=0.1
)

print("\n--- Evaluating MLP ---")

test_loss, test_acc = mlp_model.evaluate(X_test, y_test)

print(f"\nMLP Test Accuracy: {test_acc * 100:.2f}%")

sample = X_test[0]
prediction = mlp_model.predict(sample.reshape(1, 28, 28))

predicted_digit = np.argmax(prediction)

```

```

print("Actual Digit:", y_test[0])
print("Predicted Digit:", predicted_digit)

plt.imshow(sample, cmap='gray')
plt.title(f"Predicted: {predicted_digit}")
plt.axis('off')
plt.show()

plt.figure()
plt.plot(history.history['accuracy'], marker='o', label='Training Accuracy')
plt.plot(history.history['val_accuracy'], marker='o', label='Validation Accuracy')

plt.title('MLP Accuracy vs Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid()

plt.show()

plt.figure()
plt.plot(history.history['loss'], marker='o', label='Training Loss')
plt.plot(history.history['val_loss'], marker='o', label='Validation Loss')

plt.title('MLP Loss vs Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid()

plt.show()

y_pred = mlp_model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')

plt.title("Confusion Matrix")
plt.show()

```

Training Images Shape: (60000, 28, 28)
Testing Images Shape: (10000, 28, 28)

/Users/vemula/myenv/lib/python3.13/site-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/`ut\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_4 (Dense)	(None, 128)	100,480
dense_5 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

--- Training MLP ---

Epoch 1/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9206 - loss: 0.2779 - val_accuracy: 0.9655 - val_loss: 0.1248
Epoch 2/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9634 - loss: 0.1233 - val_accuracy: 0.9720 - val_loss: 0.0943
Epoch 3/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9751 - loss: 0.0841 - val_accuracy: 0.9732 - val_loss: 0.0874
Epoch 4/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9804 - loss: 0.0632 - val_accuracy: 0.9765 - val_loss: 0.0772
Epoch 5/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9853 - loss: 0.0493 - val_accuracy: 0.9785 - val_loss: 0.0718
Epoch 6/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9884 - loss: 0.0383 - val_accuracy: 0.9798 - val_loss: 0.0706
Epoch 7/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9908 - loss: 0.0301 - val_accuracy: 0.9775 - val_loss: 0.0814
Epoch 8/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9924 - loss: 0.0245 - val_accuracy: 0.9773 - val_loss: 0.0800
Epoch 9/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9945 - loss: 0.0195 - val_accuracy: 0.9817 - val_loss: 0.0734
Epoch 10/10
1688/1688 ————— 2s 1ms/step - accuracy: 0.9945 - loss: 0.0176 - val_accuracy: 0.9793 - val_loss: 0.0806

--- Evaluating MLP ---

313/313 ————— 0s 407us/step - accuracy: 0.9787 - loss: 0.0802

MLP Test Accuracy: 97.87%

1/1 ————— 0s 19ms/step

Actual Digit: 7

Predicted Digit: 7

3. Hyperparameter Experimentation Log

Run your model multiple times, varying the activation functions and learning rates. Record the final training loss and accuracy after a fixed number of epochs (e.g., 50 or 100) for each configuration.

Exp #	Activation	Learning Rate	Final Loss	Final Acc.
1	Sigmoid	0.01	0.5617	100.0

2	Sigmoid	0.1	0.0257	100.0
3	ReLU	0.01	0.5002	75.0
4	ReLU	0.1	0.3475	75.0

4. Comparative Analysis

Based on your experimental data, compare the performance of **Sigmoid** versus **ReLU**. Which activation function allowed the model to converge faster? How did the learning rate influence the stability of the loss curve?

From the experiments, the **ReLU activation function** converged faster than the **Sigmoid activation function**. ReLU reduced the training loss more quickly and reached higher accuracy in fewer epochs because it avoids the vanishing gradient problem that can affect Sigmoid. Increasing the learning rate from **0.01** to **0.1** improved the convergence speed by allowing the model to update its weights more aggressively. However, if the learning rate is set too high, the loss curve may become unstable and fail to converge properly. In this experiment, **ReLU with a learning rate of 0.1** provided the best overall performance, achieving the lowest loss and the highest accuracy.